



四川大学

四川大学

计算机科学与技术学院

操作系统实验报告

实验四：自定义内存分配器 (Malloc)

姓 名 许盛凯

学 号 2024141460445

专 业 计算机科学与技术

指导教师 左劼

2026 年 5 月 12 日

任务一：实现简单内存分配器

2.1 提交 my_malloc.c 的完整代码

Listing 1: my_malloc.c — 基础分配器完整实现

```
1  #include <unistd.h>
2  #include <string.h>
3  #include <stdio.h>
4
5  typedef struct block_meta {
6      size_t size;
7      int free;          /* 空闲为 1, 占用为 0 */
8      struct block_meta *next;
9  } block_meta;
10
11 #define META_SIZE sizeof(block_meta)
12
13 void *global_base = NULL;
14
15 /* ===== 1. 寻找空闲块 ===== */
16 block_meta *find_free_block(block_meta **last, size_t size) {
17     block_meta *current = global_base;
18     /* TODO 1: First-Fit 遍历, 同时维护 *last 指向链表尾部 */
19     while (current && !(current->free && current->size >= size))
20     {
21         *last = current;
22         current = current->next;
23     }
24     return current;
25
26 /* ===== 2. 向 OS 申请新堆空间 ===== */
27 block_meta *request_space(block_meta *last, size_t size) {
28     block_meta *block;
29     /* TODO 2: sbrk(0) 获取当前堆顶作为新块起始地址 */
30     block = sbrk(0);
31     /* TODO 3: sbrk 扩展堆, 申请失败时返回 NULL */
32     void *request = sbrk(META_SIZE + size);
33     if (request == (void *)-1)
```

```
34     return NULL;
35     /* TODO 4: 初始化元数据 */
36     block->size = size;
37     block->free = 0;
38     block->next = NULL;
39     /* TODO 5: 将新块接到链表尾部 */
40     if (last)
41         last->next = block;
42     return block;
43 }
44
45 /* ===== 3. my_malloc ===== */
46 void *my_malloc(size_t size) {
47     if (size <= 0) return NULL;
48     block_meta *block;
49     if (!global_base) {
50         /* TODO 6: 第一次分配, 初始化 global_base */
51         block = request_space(NULL, size);
52         if (!block) return NULL;
53         global_base = block;
54     } else {
55         block_meta *last = global_base;
56         /* TODO 7: 尝试复用空闲块, 找不到时扩容 */
57         block = find_free_block(&last, size);
58         if (!block) {
59             block = request_space(last, size);
60             if (!block) return NULL;
61         } else {
62             block->free = 0; /* 标记为占用 */
63         }
64     }
65     /* TODO 8: 跳过元数据头部, 返回用户数据区指针 */
66     return (block + 1);
67 }
68
69 /* ===== 4. my_free ===== */
70 void my_free(void *ptr) {
71     if (!ptr) return;
72     /* TODO 9: 反推元数据地址, 标记为空闲 */
73     block_meta *block = (block_meta *)ptr - 1;
```

```
74     block->free = 1;  
75 }
```

关键实现要点说明

1. **TODO 1** (`find_free_block`): 使用 `while` 循环遍历链表, 条件为”当前块不满足: 空闲且大小足够”, 同时将 `*last` 持续更新为最后访问的节点, 以备后续追加新块使用。
2. **TODO 2-3** (`request_space`): 先用 `sbrk(0)` 查询当前 program break (即新块的起始地址), 再用 `sbrk(META_SIZE + size)` 一次性申请元数据区 + 数据区。两次调用之间不会发生竞争 (单线程), 因此 `block` 指针始终指向新分配区域的起始位置。
3. **TODO 4-5**: 将新块的 `size` 设置为用户请求大小 (不含元数据), `free = 0`, `next = NULL`; 若 `last != NULL` 则接回链表。
4. **TODO 6-7** (`my_malloc`): 首次分配时直接调用 `request_space`; 后续分配优先复用空闲块, 否则扩容。复用时必须将 `block->free` 置 0, 避免同一块被重复返回。
5. **TODO 8**: `block + 1` 利用 C 指针算术自动偏移 `sizeof(block_meta)` 字节, 使返回地址指向用户可读写区域。
6. **TODO 9** (`my_free`): `(block_meta *)ptr - 1` 是 TODO 8 的逆操作, 从用户指针还原元数据指针, 将 `free` 设为 1 即完成释放。

2.2 基础测试运行截图



图 1: 基础测试运行截图

任务二：LD_PRELOAD 动态拦截

本任务中的 hook_malloc.c 由学生手册直接给出，此处不重复提交源码。

3.1 LD_PRELOAD 拦截运行截图



图 2: LD_PRELOAD 动态拦截运行截图

实验思考题

1. 元数据开销：自定义分配器为什么不能只记录”用户申请了多少字节”，而必须在用户数据区之外维护额外元数据？

内存分配器在管理内存时需要完成三项核心操作：知道一个块有多大（size）、知道这个块当前是否空闲（free）、知道下一个块在哪里（next）。仅记录”用户申请了多少字节”无法满足这三项需求：

- (1) **释放时需要知道块的大小。** my_free(ptr) 收到的是用户数据区指针，如果没有元数据，分配器无法知道这个块占用了多少空间，也就无法正确更新空闲链表。
- (2) **复用时需要判断块是否空闲及容量是否足够。** First-Fit 遍历链表时，必须读取每个块的 free 标志和 size 字段，这些信息只能通过元数据获取，用户数据区本身不携带任何管理信息。
- (3) **链表遍历需要 next 指针。** 堆是线性地址空间，若没有 next 指针，分配器无法从一个块跳转到下一个块，也就无法实现链表遍历。

因此，元数据是分配器维持自身状态、支持释放和复用功能的必要开销，必须独立于用户数据区之外维护，且通常紧邻在用户数据区之前，以便通过指针算术（ptr -

1) 快速定位。

2. 堆空间管理：本实验使用 `sbrk` 扩展堆空间。和每次都直接向操作系统申请一块全新内存相比，分配器自己维护空闲链表有什么意义？

(1) **避免频繁系统调用，降低开销。** `sbrk/mmap` 是系统调用，需要从用户态陷入内核，开销远大于用户态函数调用。如果每次 `malloc` 都触发系统调用，而被释放的内存又立刻还给内核，则下次申请还需再次陷入内核，效率极低。维护空闲链表后，已释放的内存可以直接在用户态复用，绝大多数 `malloc/free` 不需要系统调用。

(2) **减少内存碎片，提升内存利用率。** 操作系统以页（通常 4KB）为单位管理物理内存。若程序频繁申请小块内存（如几十字节），每次都向 OS 申请整页显然浪费。分配器将一次 `sbrk` 申请的大块内存切分给多个用户请求，同一页内可以容纳大量小块，利用率大幅提升。

(3) **支持内存复用，延长已分配内存的生命周期。** 释放的块保留在堆中等待被复用，而非立即归还操作系统。这使得“申请—释放—再申请”模式不会导致堆地址空间来回收缩扩张，program break 的移动次数显著减少，有助于减少地址空间碎片。

(4) **为上层提供统一接口。** 应用程序通过 `malloc/free` 使用内存，无需感知系统调用细节，分配器充当了用户程序与操作系统内存管理之间的缓冲层。

3. 释放语义：本实验中的 `my_free` 只把块标记为空闲，而不清空内容，也不立刻归还给操作系统。这样设计分别带来了哪些好处和风险？

好处：

(1) **性能高。** 仅修改一个 `int` 字段 (`free = 1`)，时间复杂度 $O(1)$ ，不需要 `memset`，也不需要系统调用，释放开销极小。

(2) **方便内存复用。** 已释放的块保留在堆中，下次 `my_malloc` 通过 First-Fit 策略可以直接复用，无需再次向操作系统申请，节省 `sbrk` 调用次数。

(3) **简化实现。** 不需要维护“释放后是否归还 OS”的复杂逻辑（如 `sbrk` 只能降低位于堆顶的连续空闲区域），实现更简单、可靠。

风险：

(1) **敏感数据残留。** 释放后内容不清空，若该内存被复用，新用户读到旧数据（密码、密钥、个人信息等），存在信息泄露风险。

(2) **悬空指针 (Dangling Pointer) 隐患。** 释放后原指针依然指向合法地址，且数据可能尚未被覆盖。若程序错误地继续使用已释放的指针 (use-after-free)，行为可能“看起来正常”，导致难以发现的 bug。

(3) **内存不会及时归还 OS**。长时间运行的程序如果大量释放内存，堆地址空间会持续增大，不归还给内核，其他进程无法使用这部分物理内存，可能造成系统级内存压力。

4. 动态拦截范围：真实程序可能调用 malloc/free/calloc/realloc 等多个接口。如果只替换其中一部分接口，可能会造成哪些不一致？

内存分配接口之间存在隐含的语义约定：malloc 分配的内存必须由 free 释放；calloc 分配的内存也必须由 free 释放；realloc 会在内部调用 malloc 和 free。如果只替换部分接口，会产生以下不一致：

(1) **释放器与分配器不匹配导致崩溃**。若只替换 malloc (使用自定义堆)，而不替换 free (仍然使用 libc 的 free)，则 libc 的 free 会尝试按照 libc 的内存布局解析元数据，但自定义堆的 block_meta 结构与 libc 不同，导致非法内存访问或程序崩溃。

(2) **realloc 无法正确处理跨分配器的指针**。若只替换 malloc 而不替换 realloc，libc 的 realloc 可能对自定义分配器分配的指针执行复制和释放操作，由于元数据格式不兼容，行为未定义。

(3) **calloc 分配的内存被自定义 free 误释放**。反之，若 calloc 使用 libc 实现而 free 被替换为自定义版本，自定义 free 会用 $\text{ptr} - 1$ 的方式读取元数据，而 libc calloc 分配的内存并没有此格式的元数据头部，同样导致错误。

(4) **统计/调试数据不完整**。若只拦截 malloc 进行调试计数，而 calloc/realloc 未被拦截，则统计到的内存分配次数偏低，无法准确反映程序的内存行为。

因此，实际工程中替换内存分配接口时，必须同时替换所有相关接口，确保“分配”与“释放”始终经由同一套分配器。

5. 真实分配器复杂性：和本实验的简化分配器相比，生产环境中的 malloc 还需要重点解决哪些问题？请选择两点说明原因。

第一点：线程安全 (Thread Safety)

本实验的 global_base 是一个全局变量，find_free_block、request_space、my_malloc、my_free 都会修改链表结构，均存在竞态条件 (Race Condition)。在多线程程序中，若两个线程同时调用 my_malloc，可能发生：两个线程同时读到 global_base == NULL，各自调用 request_space，导致 global_base 被初始化两次（只有最后一次赋值有效），其中一次 sbrk 分配的内存区域丢失引用。此外链表的 next 指针也可能被并发读写破坏。

生产级 malloc (如 glibc 的 ptmalloc2、Google 的 tcmalloc) 通过细粒度锁或每线程独立 arena 等机制解决此问题。ptmalloc2 为每个线程分配独立的 arena (内存池)，

减少锁竞争；tcmalloc 更进一步，为每个线程维护本地缓存，小对象分配完全无锁，大幅提升多线程吞吐量。

第二点：内存对齐 (Alignment)

本实验直接用 `sbrk(0)` 的返回值作为新块地址，没有保证返回给用户的地址满足特定对齐要求。在现代 x86-64 体系结构中，许多数据类型（如 `double`、SIMD 向量 `__m128`）要求 8 字节或 16 字节对齐；若分配的内存地址未对齐，轻则性能下降（非对齐访问触发额外内存总线操作），重则在某些架构（如早期 ARM、SPARC）上引发总线错误（Bus Error）导致程序崩溃。

C 标准要求 `malloc` 返回的地址对任何基本类型都满足对齐要求，通常为 8 或 16 字节对齐。生产级分配器在返回指针前会将地址向上对齐（round up），并在元数据大小和用户数据大小计算中同时考虑对齐需求，以保证正确性和性能。

进阶挑战（二选一）

选择挑战 ✓ 挑战一：块的分割 (Splitting)

完成情况 ✓ 已完成

5.1 核心实现代码

Listing 2: 新增的 `split_block` 函数及修改后的 `my_malloc` 关键部分

```
1  /* ===== 进阶挑战一：块分割 (Splitting) ===== */
2
3  /**
4   * split_block: 将一个大空闲块切分为两个块。
5   *   block       - 即将被复用的空闲块（已找到，大小足够）
6   *   size        - 本次用户实际请求大小
7   *
8   * 只有当剩余空间足以容纳新的 block_meta + 至少 1 字节数据时才执行分割，
9   * 否则不值得引入新节点（内碎片损耗更小）。
10  */
11 void split_block(block_meta *block, size_t size) {
12     if (block->size >= size + META_SIZE + 1) {
13         /* 新空闲块的元数据位于当前块数据区偏移 size 处 */
14         block_meta *new_block =
15             (block_meta *)((char *) (block + 1) + size);
16     }
```

```
17     new_block->size = block->size - size - META_SIZE;
18     new_block->free = 1;
19     new_block->next = block->next; /* 继承原 next */
20
21     /* 收缩当前块，并接入新空闲块 */
22     block->size = size;
23     block->next = new_block;
24 }
25 /* 若剩余不足，不分割，整块给用户（保持原有逻辑） */
26 }
27
28 /* ===== 修改后的 my_malloc 关键路径 ===== */
29 void *my_malloc(size_t size) {
30     if (size <= 0) return NULL;
31     block_meta *block;
32     if (!global_base) {
33         block = request_space(NULL, size);
34         if (!block) return NULL;
35         global_base = block;
36     } else {
37         block_meta *last = global_base;
38         block = find_free_block(&last, size);
39         if (!block) {
40             block = request_space(last, size);
41             if (!block) return NULL;
42         } else {
43             split_block(block, size); /* 【新增】分割大空闲块 */
44             block->free = 0;
45         }
46     }
47     return (block + 1);
48 }
```

5.2 设计说明

问题背景 (内部碎片): 基础实现中, 当一个 200 字节的空闲块被 `my_malloc(10)` 复用后, 整个 200 字节都被标记为”正在使用”, 但用户只需要 10 字节, 剩余 190 字节成为无法再分配的内部碎片。

核心思路: 在找到可复用的空闲块、将其标记为占用之前, 先检查该块是否远大于本次请求。如果块的剩余空间足够放下一个新的 `block_meta` 结构 (`META_SIZE`) 加上至少 1 字节的用户数据, 就在当前块数据区的 `size` 偏移处就地构造一个新的空闲块, 插入链表。

关键判断条件:

$$\text{block} \rightarrow \text{size} \geq \text{size} + \text{META_SIZE} + 1$$

这个条件确保分割后的”尾部块”至少有 1 字节可用数据区, 避免生成无意义的零大小块。

链表维护:

- `new_block->next = block->next`: 新块继承原块的后继指针, 保持链表连通。
- `block->next = new_block`: 当前块的后继更新为新空闲块, 将其插入链表。
- `block->size = size`: 当前块大小精确裁剪为用户请求大小, 不多分配。

碎片减少效果: 分割后, 原本 200 字节的大块变为”10 字节 (使用中) + `META_SIZE` + 190 - `META_SIZE` 字节 (空闲)”。后续再次申请不超过剩余大小的内存时, 可以直接复用”尾部空闲块”, 无需扩展堆, 显著减少了内部碎片和 `sbrk` 调用次数。